

Esercizi svolti

Assembler ARMv7 per AESO

Prima parte

Pattern per la compilazione di comandi

Compilazione di un if-then Pattern per la compilazione di uno statement

```
if(condizione) then Comandi
resto del codice
```

```
    .... codice che setta i flag per il test della condizione
    B(not Condizione) cont
    codice risultato della compilazione di Comandi
cont:
    codice risultato dalla compilazione del resto del codice
```

Compilazione if-then-else Pattern per la compilazione di uno statemet

```
if(condizione) then RamoThen else RamoElse
resto del codice
```

```
    ... codice che setta i flag per il test della condizione
    B(Condizione) then
else:
    compilazione del codice RamoElse
    B cont
then:
    compilazione del codice RamoThen
cont:
    compilazione del resto del codice
```

Compilazione di un for loop Pattern per la compilazione di uno statement

```
for(init; termcond; iterinc)
    corpo del forloop
resto del codice
```

```
    init    @ inizilalizzazione della variabile di iterazione
loop:    calcolo termcond
    B(termcond true) fine
    compilazione del corpo del forloop
    iterinc @ calcolo del nuovo valore della var di iterazione
    b loop
```

compilazione del resto del codice

compilazione di una chiamata di funzione o procedura Pattern per la compilazione di una chiamata tipo

```
ret = fun(p1, p2)
resto del codice
```

```
push {LR}          @ salvare un eventuale LR ancora da utilizzare
mov r0, ...p1.. @ o calcolo di p1 in R0 con altre istruzioni (add r0, r1, r2)
mov r1, ...p2.. @ idem
bl fun
mov ..., R0         @ utilizza il risultato della funzione (oppure usa R0 come sorgente)
compilazione del resto del codice
```

compilazione di una funzione Pattern per la compilazione di una funzione tipo

```
int fun(int a, int b) {
    corpo della funzione
    return(x);
}
```

```
.global fun
.type fun, %function
fun:
    push {r4-r11,LR}    @ solo in caso di utilizzo dei registri da r4 a r11 (anche non tutti)

    compilazione del corpo della funzione @ usa a in R0 e b in R1
    mov R0, ...x...     @ sposta il valore da restituire in R0

    pop { .... }        @ ripristino di tutti e solo i registri salvati in precedenza sullo stack
    mov PC, LR @ può essere sostituita da una pop {r4-r11,PC}
                    @ tutti i registri salvati e il LR viene ripescato come PC
```

compilazione di un main Pattern per la compilazione di un main

```
.text
.global main
.type main, %function

int main(int argc, char ** argv) {
    ...
}
```

```

main:      @ r0 è argv
           @ r1 è il puntatore al primo argv (stringa di car null terminated)
           @ il prossimo argv è [r1,#4]
           @ per accedere al puntatore del primo argomento:
           @ ldr r2,[r1]
           @ ldr r2, [r2]

           push {lr}   @ solo se utilizza nel corpo del main una BL
compilazione del corpo del main
           pop {lr}    @ solo se precedentemente salvato
           mov pc, lr  @ ritorno al chiamante (S.O.)

```

Dichiarazioni e variabili

Variabili scalari

Per utilizzare una variabile scalare di un tipo primitivo, utilizziamo di norma i registri. Una sezione di codice tipo

```
int x = 123;
```

```
x++;
```

considera di utilizzare un registro per x (per esempio r3) e poi lo usa di conseguenza

```
mov r3, #123
add r3, r3, #1
```

Vettori

Per i vettori normalmente utilizziamo zone di memoria contigue, a partire da un indirizzo definito da una etichetta (nome del vettore) che permette di recuperare l'indirizzo di partenza in memoria mediante una **ldr Rx, =etichetta**. L'area di memoria può essere dichiarata utilizzando una **.space**

Ad esempio, la dichiarazione di un vettore di interi

```
int x[16];
```

può essere implementata con una sezione

```
.data
x: .space 16
```

e successivamente l'elemento i-esimo può essere acceduto (supponendo di avere i nel registro r0) in lettura con il codice

```
ldr r1, =x @ base
ldr r2, [r1, r0, lsl #2] @ base più indice (moltiplicato per il sizeof(int))
```

mentre per l'accesso in scrittura utilizzeremo il codice

```
ldr r1, =x @ base
str r2, [r1, r0, lsl #2] @ base più indice (moltiplicato per il sizeof(int))
```

per assegnare a x[i] il valore contenuto in R2.

struct

Per le strutture, riserviamo un'area di memoria pari alla sizeof della struttura. Successivamente gli elementi della struttura li possiamo accedere con indirizzamento base+indice dove l'indice viene utilizzato come offset (in byte) dalla base della struct. Per esempio:

```
struct {
    int i;
    int j;
    char * s;
} A;
```

può essere implementata così

```
.data
A: .space 3
```

e successivamente, volendo leggere in r4, r5 e r6 i, j e l'indirizzo s potremmo usare il codice

```
ldr r0, =A @ caricamento dell'indirizzo base della struttura

ldr r4, [r0] @ carico i in r4
ldr r5, [r0, #4] @ carico j in r5
ldr r6, [r0, #8] @ carico l'indirizzo della stringa in r6
```

variabili “pemanenti”

Le variabili in memoria, accedibili dall'ambiente “globale” in realtà sono di solito implementate con allocato nella sezione data. In questo l'accesso alla variabile deve avvenire mediante LDR (accesso in lettura) e STR (accesso in scrittura).

Tipicamente, se avessimo una situazione tipo questa:

```
int x = 123;

int main(int argc, char **argv) {

    x = x / 2;
```

dovremmo pensare di implementarla con un codice come quello che segue:

```
.text

main:

    ldr r1, =x;           @indirizzo base di x in r0

    ldr r0, [r1]           @ valore di x in r0
    asr 0, r0, #1          @ shift a destra di una posizione => divisione per 2
    str r0, [r1]           @ modifica della memoria
```

ottimizzazioni Quando si utilizzano valori che debbono essere conservati in memoria, in presenza di aggiornamenti successivi, si può caricare il valore iniziale dalla memoria una sola volta e poi memorizzare solo il valore assegnato alla fine degli aggiornamenti. Per esempio, in presenza di un codice

```
for(i=0; i<n; i++) {
    sum = sum + x[i];
}
```

(assumendo che x sia un valore memorizzato all'indirizzo dell'etichetta **x**) dovrebbe essere compilato in un codice tipo quello che segue:

```
    ldr r0, =x             @ indirizzo di x
    ldr r1, =sum            @ indirizzo di sum

    mov r3, #0             @ i
loop:  cmp r3, #16          @ n = 16 (per esempio)
    beq fineloop
    ldr r4, [r0, r3, lsl #2] @ carico x[i]
    ldr r2, [r1]            @ valore di sum in r2
    add r2, r2, r4          @ sum = sum + x[i]
    str r2, [r1]            @ sum memorizzato
    add r3, r3, #1          @ i++
    b loop
fine:
```

tuttavia due iterazioni successive provocherebbero la scrittura del valore di sum della prima iterazione e poi il recupero di quel valore per aggiornarlo nella seconda iterazione. In buona sostanza, avremmo una

```
ldr r2, [r1]
```

seguita da una

```
str r2, [r1]
```

nella prima iterazione, seguita poi da un'altra

```
ldr r2, [r1]
```

nella seconda iterazione. Cio' è assolutamente inutile. possiamo eliminare la *str*, eseguire la *ldr* solo prima dell'inizio delle iterazioni e scrivere il risultato in memoria per la variabile *sum* solo quando il ciclo è terminato:

```
    ldr r0, =x      @ indirizzo di x
    ldr r1, =sum     @ indirizzo di sum
    ldr r2, [r1]     @ valore iniziale di sum in r2
    mov r3, #0       @ i = 0

loop:  cmp r3, #16 @ n = 16 (per esempio)
       beq fineloop
       ldr r4, [r0, r3, lsl #2] @ carico x[i]
       add r2, r2, r4 @ sum = sum + x[i]
       add r3, r3, #1 @ i++
       b loop
fine:  @ solo quando è finito il ciclo degli aggiornamenti ...
       str r2, [r1] @ memorizzo sum
```

Sezioni nel codice

Tutto il codice deve essere in una sezione testo che inizia con la pseudo istruzione

```
.text
```

I dati vanno nella sezione che inizia con la pseudo istruzione

```
.data
```

Pseudo istruzioni per la sezione dati

parole

Si può indicare nella sezione *.data*

```
etichetta: .word w1,w2, ...
```

All'indirizzo che può essere ottenuto in *Ri* con una

```
ldr Ri, =etichetta
```

riserva tante parole quanti sono gli argomenti della *.word* e assegna loro i valori *w1*, *w2*, ...

Una volta caricato *Ri* con l'indirizzo di etichetta, si possono accedere le parole con

```
ldr Rx, [Ri]          @ valore w1 in Rx
ldr Ry, [Ri, #4]      @ valore w2 in Ry
etc
```

oppure, se Rj è inizializzato a 0 con

```
ldr Rx, [Ri, Rj, lsl #2]
add Rj, Rj, #1
ldr Ry, [Ri, Rj, lsl #2]
etc
```

oppure, se non interessa preservare il valore Ri

```
ldr Rx, [Ri], #4
ldr Ry, [Ri], #4
etc
```

byte

Come per le parole, possiamo utilizzare

```
etichetta: .byte b1, b2, ...
```

per riservare un'area di memoria che contiene i valori b1, b2, ... (ciascuno in un byte, con indirizzi consecutivi)

stringhe

```
etichetta: .string "abcdef"
```

riserva un'area di memoria all'indirizzo *etichetta* di tanti byte quanti ne sono necessari per contenere la stringa, compreso il NULL alla fine (byte = 0)

spazio

```
etichetta: .space nnn
```

riserva un'area di memoria all'indirizzo "etichetta" di nnn parole

Esercizi su compilazione di pezzi di codice C

compilazione di if-then Compilare il codice C

```
if(x < 0) x = -x;
```

E' uno statement condizionale. Per testare la condizione confrontiamo x con 0. Poi la condizione LT (minore di) ci dirà se x è negativo. La compilazione di un if-then prevede che utilizziamo la condizione inversa per saltare il codice del ramo then. Dunque come condizione per saltare il ramo then dovremo utilizzare una GE (maggiore o uguale). Infine, per la compilazione del ramo then dobbiamo calcolare il complemento a due di x. Cosa che si fa invertendo tutti i bit e sommando 1.

Il codice potrebbe essere:

```
.text
.global main
.type main, %function

main:
    ldr r0, =x
    ldr r0, [r0]    @ x in r0

    @@@@@@@ la parte che compila lo statement if then comincia qui
    cmp r0, #0      @ confronta x con 0
    bge cont        @ se non è minore salta il codice del ramo then
then:               @ etichetta di comodo, giusto per evidenziare il ramo then
    mvn r0, r0      @ negazione bit a bit di r0 in r0
    add r0, r0, #1   @ somma 1, ottieni complemento a due cioè -n
    @@@@@@@ e termina qui
cont:
    mov pc, lr

.data
x: .word -12
```

compilazione di ciclo for

Supponiamo di dover compilare il codice

```
for(i=0; i<n; i++)
    d[i] = c[i];          /* d e c vettori di byte (char) */
```

Questo possiamo farlo con il codice:

```
.text
.global main
.type main, %function

main:
    ldr r0, =s
    ldr r1, =d
    mov r2, #14 @ lunghezza della prima stringa

    @@@@@@@ codice compilato comincia qui
for:   ldrb r3, [r0],#1    @ s[i] in r3 e predisponi r0 a puntare al prossimo char
    cmp r3, #0            @ è un NULL ?
    strb r3, [r1],#1      @ comunque copialo nella stringa destinazione (e predisponi
                          @ r2 a puntare al prossimo byte nel buffer destinazione
```



```

    beq fine      @ e se era null, termina
    b for         @ altrimenti copia un altro carattere
fine:
    @@@@@@@@@ e finisce qui

    mov pc, lr

.data
s: .string "ciao come va?"
d: .string "          "

```

compilazione di do-while

Il comando `do { ... } while(c);` esegue gli statement ... finché la condizione `c` è vera. Supponiamo di compilare il codice che segue, che calcola il risultato di una divisione fra interi (nell'ipotesi che il dividendo non sia minore del divisore):

```

res = 0;
do { a = a - b;  res = res + 1; } while(a >= b);

```

Il codice compilato in assembler sarà

```

    mov r2, #0 @ res in r2, a in r0, b in r1
do:   sub r0, r0, r1 @ a = a - b
      add r2, r2, #1 @ res = res + 1

      cmp r0, r1 @ calcolo della condizione
      bge do     @ se vera esegui un'altra iterazione

fine: @ altrimenti fine

```

chiamata di funzioni diversi a seconda di un parametro

Codice C:

```

if(p == 0) res = f(x);
if(p == 1) res = g(x);
if(p == 2) res = h(x);

```

supponendo di avere il valore di `p` in `r4`, `res` in `r5` e `x` in `r6`, il codice sarà

```

    cmp r4, #0
    moveq r0, r6
    bleq f
    moveq r5, r0
    cmp r4, #1
    moveq r0, r6
    bleq g

```

```

moveq r5, r0
cmp r4, #2
moveq r0, r6
bleq h
moveq r5, r0

```

Seconda parte (esercizi)

Calcolo di semplice funzione

Implementare una funzione che calcola

$\text{fun}(x) = (x * 3 + 1) / 4$

Il codice è il seguente:

```

fun:  lsl r1, r0, #1  @ x*2
      add r1, r1, r0  @ x*2 + x = 3x
      add r1, r1, #1  @ 3x+1
      asr r0, r1, #2  @ /4
      mov pc, lr     @ e ritorna

```

Controllare se un numero da 32 bit è una potenza di 2

Utilizziamo una funzione che conta il numero dei bit a 1 in un registro:

```

.text
.global conta
.type conta, %function

conta:
    mov r1, #0          @ contatore, inizializzato a 1
    mov r2, #32         @ numero dei bit nel registro
loop:  ands r3, r0, #1   @ test sul primo bit, risultato si scarta
      addne r1, r1, #1   @ se il risultato non è 0, il bit era a 1, incrementa contatore
      lsr r0, r0, #1     @ scarta il bit testato
      subs r2, r2, #1    @ decrementa il numero di bit da testare
      bne loop          @ esegui un'altra iterazione
end:   mov r0, r1        @ quando hai finito, sposta il contatore nel registro r0
      mov pc, lr        @ e torna al chiamante

```

Successivamente possiamo utilizzare la funzione per vedere se c'è un unico bit settato a 1 nel registro passato come argomento:

```

.text
.global pot
.type pot, %function

```

```

pot:  push {lr}          @ salva il link register perchè uso una BL
      bl conta          @ conta il numero di 1 nel registro
      cmp r0, #1        @ controlla se è 1
      movne r0, #0      @ se non è 1, restituisci 0 (false)
si:   pop {pc}          @ altrimenti ritorna al chiamante con 1 (true)

```

Per testare il codice, utilizziamo un main che legge gli argomenti dalla riga di comando e stampa un messaggio che dice se ognuno degli argomenti è o non è una potenza di 2.

```

.text
.global main
.type main, %function

main:  push {r4,lr}
      add r0, r1, #4 @ argv 1
      ldr r0, [r0]
      bl atoi
      mov r4, r0
      bl pot
      cmp r0, #1
      ldreq r2, =si
      ldrne r2, =no
      mov r1, r4
      ldr r0, =fmt
      bl printf
      pop {r4,pc}

.data
fmt:   .string "Il numero %d %s e' una potenza di due\n"
si:    .string "      "
no:    .string " non "

```

Contare il numero di occorrenze di ciascun carattere in una stringa

Il codice controlla tutti i caratteri alfanumerici. Se sono maiuscole, le rende minuscole in modo che 'a' e 'A' contino per la stessa lettera.

Alla fine restituisce un vettore che contiene in ciascuna posizione il numero di occorrenze del carattere che ha come posizione nell'alfabeto l'indice del vettore. Posizione 0 -> lettera A, posizione 1 -> lettera B, etc.

```

.text
.global conta
.type conta, %function

@ r0 indirizzo della stringa

```

```

conta:    push {r4-r6,lr}      @ salva non temporanei utilizzati e ind ret
          mov r4, r0           @ indirizzo della stringa
          mov r0, #26          @ alloca spazio per 26 interi
          lsl r0, #2           @ da 4 byte
          bl malloc            @ alloca la memoria
          mov r5, r0           @ indirizzo vettore (sarebbe da controllare)
          mov r1, #26          @ numero di iterazioni init
          mov r2, #0           @ valore da assegnare agli elem vettore
init:     cmp r1, #0           @ sono alla fine?
          beq finei            @ smetti
          str r2, [r0], #4     @ v[i] = 0
          sub r1, #1           @ conta--
          b init               @ altro giro
finei:    ldrb r0, [r4], #1     @ carica carattere
          cmp r0, #0           @ se e' null
          beq fines            @ finelavori
          bl getcode           @ restituisce codice del carattere o -1
          cmp r0, #-1          @ se non era alfabetico
          beq finei            @ ricomincia col prossimo
          ldr r1, [r5, r0, lsl #2] @ v[c]
          add r1, r1, #1        @ += 1
          str r1, [r5, r0, lsl #2] @ e salva v[c]
          b finei              @ altro carattere
fines:    mov r0, r5            @ restituisci indirizzo vettore
          pop {r4-r6,pc}       @ return

getcode:
          cmp r0, #'A'          @ controlla se viene prima della A
          blt non               @ se prima => non lettera
          cmp r0, #'Z'          @ se prima di Z allora è maiuscola
          ble maiusc
          cmp r0, #'a'          @ controlla se prima della a
          blt non               @ se prima => allora non lettera
          cmp r0, #'z'          @ se dopo la z
          bgt non               @ non lettera
minusc:   sub r0, r0, #'a'      @ altimenti minuscola
          mov pc, lr
maiusc:   sub r0, r0, #'A'      @ maiuscola
          mov pc, lr
non:      mov r0, #-1
          mov pc, lr

```

Per provare il funzionamento del programma:

```
#include <stdio.h>
```

```
char * s = "questa e una prova di stringa non tanto lunga scritta in italiano. mi piacerebb
```

```

extern int * (conta) (char *);

int main(int argc, char ** argv) {

    int * v;
    if(argc==1) v = conta(s);
    else v = conta(argv[1]);

    for(int i=0; i<26; i++)
        printf("v[%c] = %d\n",
               (i+'a'),v[i]);
    return(0);
}

```

Ricerca in un vettore di interi (non ordinato)

La funzione *cerca* verifica se un certo numero di trova in un vettore di interi di lunghezza nota. La signature è:

```
int cerca(int x, int *v, int n)
```

La funzione *cerca* implementata in assembler ha questo codice:

```

.text
.global cerca
.type cerca, %function

    @r0 x r1 v r2 n
cerca: push {r4}
      mov r3, #0  @i
loop:  ldr r4, [r1, r3, lsl #2]
      cmp r4, r0
      beq trovato
      add r3, r3, #1
      cmp r3, r2
      beq nontrovato
      b loop
trovato:mov r0, r3
        b cont
nontrovato: mov r0, #-1
cont:  pop {r4}
      mov pc, lr

```

Il programma C che segue, genera un vettore di numeri casuali e controlla se un certo numero c'è o no utilizzando la funzione *cerca*

```

#include <stdio.h>
#include <stdlib.h>

extern int cerca(int x, int *v, int n);

int main(int argc, char ** argv) {

    int n = atoi(argv[1]);           /* lunghezza vettore */
    int x = atoi(argv[2]);           /* elemento da cercare */
    int * v = (int *) malloc(sizeof(int)*n); /* allocazione vettore */
    int i, res;

    srand(123);
    for(i=0; i<n; i++) v[i] = rand() % n; /* inizializzazione random */
    printf("Cerco %d nel vettore: \n",x);
    for(i=0; i<n; i++)                /* stampa vettore */
        printf("%d ",v[i]);
    printf("\n");

    res = cerca(x, v, n);
    if(res != -1)
        printf("%d trovato alla posizione %d\n",x,res);
    else
        printf("L'elemento %d non è presente nel vettore\n",x);
    return(0);
}

```

Calcolo del fattoriale di un numero

Lo calcoliamo con una funzione ricorsiva: $\text{fact}(n) = n * \text{fact}(n-1)$. Prima di chiamare la funzione ricorsivamente, vanno salvati il valore di ritorno (LR) e il valore n sullo stack, in modo che al ritorno possiamo moltiplicare il valore restituito in $R0$ ($\text{fact}(n-1)$) per n e restituirlo al chiamante come risultato.

Il codice della funzione è il seguente

```

    @ riusciamo a fare tutto coi registri temporanei, dunque non occorre
    @ usare r4-r12, che andrebbero salvati anche loro sullo stack e
    @ ripristinati prima di tornare ...

    @ questo è l'entry point della proc ricorsiva
fact:  cmp r0, #1  @ controlla che non sia il caso base (fact(1) = 1)
      bne ric      @ se non caso base, esegui chiamata ricorsiva
      mov PC, LR   @ altrimenti restituisci 1 (e' già in R0)

ric:   push {r0,LR} @ salva n e indirizzo di ritorno
      sub r0, r0, #1 @ calcola n-1

```

```

    bl fact      @ chiamata ricorsiva (fact(n-1)) -> R0
    pop {r1,LR} @ ripristina n e indirizzo di ritorno (n -> r1 !!)
    mul r0, r1, r0 @ calcola n * fact(n-1) -> R0
    mov PC, LR   @ e ritorna (restituisce R0 come valore)

```

Per testarlo:

```

.data
fmt:  .string "Il fattoriale vale %d\n"
.text
.global main

@ indirizzo di argv in r1
@ primo parametro in argv[1] ovvero ldr _, [r1, #4]

main:  push {lr}
      ldr r0, [r1, #4]
      bl atoi

      bl fact      @ chiamata di funzione
      mov r1, r0
      ldr r0, =fmt
      bl printf
      pop {PC}     @ ritorno al chiamante

```

Fibonacci

Funzione per il calcolo ricorsivo dei numeri di Fibonacci

```

.text
.global fibo

fibo:  cmp r0, #1 @ se siamo al caso base
      movle pc, lr @ restituisco direttamente r0
      @ fib(0) = 0, fib(1) = 1
      push {lr} @ salvo indirizzo di ritorno
      sub r0, r0, #1 @ calcolo n-1
      push {r0} @ salvo (n-1)
      bl fibo @ calcolo fib(n-1)
      pop {r1} @ riprendo n-1
      push {r0} @ salvo fact(n-1)
      sub r0, r1, #1 @ calcolo n-2
      bl fibo @ chiamo fib(n-2)
      pop {r1} @ riprendo fib(n-1)
      add r0, r0, r1 @ calcolo fib(n-2)+fib(n-1)

```

```
    pop {pc}    @ e ritorno al chiamante
```

Codice per il test del programma: calcola il numero di fibonacci dell'intero passato come parametro da riga di comando:

```
.data
buf:    .fill 1024

.text
.global main

main:   push {lr}
        ldr r0, [r1, #4]    @ indirizzo base di argv[1]
        @ldr r0, =buf
        @bl gets
        @ldr r0, =buf
        bl atoi
        @mov r0, #6
        push {r0}
        bl fibo
        mov r2, r0
        pop {r1}
        ldr r0, =fmt
        bl printf
        pop {pc}

.data
fmt:    .string "Fibo(%d) = %d\n"
```

Calcolare la somma delle cifre decimali che appaiono in una stringa ASCII

Data una stringa, si vuole calcolare la somma di tutte le singole cifre numeriche. Per esempio, la stringa “a23b4bc100” deve dare 10 (2+3+4+1+0+0).

Codice asm:

```
.text
.global sommaInteri
.type sommaInteri, %function

@ indirizzo della string
sommaInteri:
    mov r1, r0 @ stringa
    mov r0, #0 @ contatore (pronto per ret)
loop:  ldrb r2, [r1], #1 @ legge carattere
        cmp r2, #0 @ se e' NULL
        moveq pc, lr @ fine lavori
```



```

sub r2,r2,'#0' @ toglilo 0
cmp r2, #9 @ se minore uguale e' numero
bgt loop @ altro carattere
add r0, r0, r2 @ e sommallo
b loop @ altro carattere

```

da notare:

- utilizziamo r0 come contatore, in modo da avere già il risultato nel registro giusto al momento del ritorno
- usiamo una costante `#'0'` che ha il significato di valere quanto il codice del carattere 0
- per verificare se un carattere è un carattere numerico, si sottrae il valore del codice dello `'0'` e si controlla che rimanga qualcosa ≤ 9

atoi

Trasforma una stringa di caratteri numerici nel corrispondente numero binario. Restituisci -1 in caso la stringa non sia formata esclusivamente da numeri.

Codice:

```

.text
.global myatoi
.type myatoi, %function

myatoi:  mov r1, #0 @ risultato
          mov r3, #10 @ costante per moltiplicazione
loop:     ldrb r2, [r0], #1 @ leggi primo char e post incrementa base
          cmp r2, #0 @ controlla NULL
          beq bene
          cmp r2,'#0' @ controlla con il codice dello '0'
          blt male @ in caso restituisco -1
          cmp r2, #'9'
          bgt male
          mul r1, r1, r3 @ shift di una posizione
          sub r2, r2,'#0' @ toglilo '0'
          add r1, r1, r2 @ aggiungi valore corrente
          b loop @ continua col prossimo carattere
male:     mov r0, #-1
          mov pc, lr
bene:     mov r0, r1
          mov pc, lr

```

calcolo funzione con parametri sullo stack

Realizzare una procedura che calcola il polinomio

$ax^2 + bx + c$

con i parametri passati tutti sullo stack:

x a b c ed indirizzo di ritorno

i parametri sono stati pushati nell'ordine ret -> x (x top dello stack) e si richiede che all'uscita della procedura lo stack contenga solo il valore calcolato. A parte il passaggio dei parametri, l'uso dei registri generali deve avvenire rispettando le solite regole tipiche di ARMv7.

Si fornisca il codice della procedura, cercando di minimizzarne il numero di istruzioni.

Il codice è quello che segue:

```
.text

.global funzione
.type funzione, %function

funzione:
    ldr r0, [sp]           @ x
    ldr r1, [sp,#8]        @ b
    ldr r3, [sp,#12]       @ c
    mul r1, r1, r0         @ bx
    add r3, r3, r1         @ bx + c
    mul r0, r0, r0         @ x^2
    ldr r1, [sp,#4]        @ a
    mul r0, r0, r1         @ a x^2
    add r0, r3, r0         @ risultato
    ldr r1, [sp, #16]      @ indirizzo di ritorno
    str r0, [sp, #16]      @ risultato sullo stack
    add sp, sp, #16        @ scarto param, risistemando lo sp (come se facessi 4 pop)
    mov pc, r1            @ ritorno
```

Nota bene: nella funzione accediamo ai parametri utilizzando come base lo stack pointer e come offset la distanza dalla testa dello stack. Prima di ritornare,

trasposta di una matrice

Il codice che segue calcola la trasposta di una matrice, memorizzata per righe.

- *swap* è la funzione che scambia le posizioni i,j e j,i
- *transpose* è la funzione che con due for scambia tutte le posizioni di una matrice
- *main.c* è il programma di prova

Funzione *swap*

```
.text
```

```

.global swap
.type swap, %function

@ r0 primo indirizzo
@ r1 secondo indirizzo

swap:  ldr r2, [r0]
       ldr r3, [r1]
       str r3, [r0]
       str r2, [r1]
       mov pc, lr

```

Funzione *transpose*

```

.text
.global transpose
.type transpose, %function

@ parametri della funzione:
@ r0 indirizzo base matrice in memoria
@ r1 n

transpose:
    push {r4-r7,lr}      @ salvataggio non temporanei usati
    mov r4, r0            @ indirizzo base
    mov r5, r1            @ n
    mov r6, #0            @ i
    mov r7, #1            @ j

loopi:    cmp r6, r5      @ i < n
    beq fine             @ in caso esci : finito trasposta
    add r7, r6, #1       @ j = i + 1

loopj:    cmp r7, r5      @ j < n
    bge finej           @ in caso i++

    mul r0, r6, r5       @ calcolo indirizzo a[i][j]
    add r0, r7, r0       @ base + i*N + j
    add r0, r4, r0, lsl #2 @ sono int quindi 4*(i*N+j)

    mul r1, r7, r5       @ calcolo indirizzo a[j][i]
    add r1, r6, r1       @ idem come sopra ma
    add r1, r4, r1, lsl #2 @ base + j*N+ i

    bl swap              @ swap

    add r7, r7, #1       @ j++

```

```

        b loopj          @ nuova iterazione j

finej:  add r6, r6, #1     @ nuova iterazione i
        b loopi

fine:   pop {r4-r7,pc}    @ ritorno

```

map su lista

Una lista è rappresentata in memoria con una parola che contiene un valore intero e la parola successiva che contiene il puntatore al prossimo elemento della lista oppure 0 se non ce ne sono altri.

Scrivere una funzione che, assumendo di avere a disposizione una funzione

```
int fun(int x)
```

applica a tutti gli elementi della lista la funzione sul loro campo valore. La funzione map prende come unico parametro l'indirizzo del primo elemento della lista, che può anche essere NULL (lista vuota).

Il codice della funzione è il seguente:

```

        .text
        .global mapl
        .type mapl, %function

mapl:    @ r0 puntatore alla lista
        cmp r0, #0 @ controlla se fosse null
        moveq pc, lr @ se null, ritorna (la somma e' gia' 0)
        push {r4,lr} @ salva i registri non temporanei utilizzati nel codice e indirizzo di r
        mov r4, r0 @ salva l'indirizzo dell'elemento in R4 (r0 sarà riscritto nella call a fun)
loop:    ldr r0, [r4] @ carica in r0 il valore dell'elemento della lista puntato da r4
        bl fun @ chiama la funzione fun
        str r0, [r4] @ sostituisci il valore dell'elemento (x) con fun(x) appena calcolato
        ldr r4, [r4,#4] @ carica in r4 il next
        cmp r4, #0 @ controlla che non sia null
        bne loop @ se è significativo, esegui un'altra volta il loop
        pop {r4,pc} @ altrimenti ripristina i registri e ritorna

```

Se l'indirizzo della funzione da chiamare fosse stato passato come secondo parametro, avremmo dovuto salvarlo in un altro registro non temporaneo

```
mov r5, r1          @ indirizzo della funzione da chiamare
```

e poi implementare la BL (che di suo non può accettare come parametro un registro) con le due istruzioni

```

mov lr, pc          @ pc in lettura viene preso dal register file e vale PC attuale +8
mov pc, R5          @ prossima istruzione da eseguire all'indirizzo di R5

```

che insieme avrebbero avuto lo stesso effetto di una ipotetica

BL R5

Controllo condizioni di bernstein

Si supponga che abbiamo a disposizione due bitmap da 32 bit che rappresentano i registri letti e scritti da due istruzioni.

- la parte alta (16 bit più significativi) contiene la bitmap dei registri letti. Il bit più significativo rappresenta il PC, quello meno significativo R0.
- la parte bassa (16 bit meno significativi) rappresenta la bitmap dei registri scritti (con le stesse convenzioni).

Si vuole implementare una funzione che controlli se le due istruzioni rispettano le condizioni di Bernstein, ovvero:

- la prima istruzione non scrive i registri letti dalla seconda (dipendenza)
- la seconda istruzione non scrive i registri letti dalla prima (anti dipendenza)
- la prima e la seconda istruzioni non scrivono lo stesso registro (dipendenza di output)

Codice della funzione:

```
.text
.global bernstein
.type bernstein, %function

bernstein:    @ r0 read/write prima istruzione
              @ r1 read/write seconda istruzione

              push {r4-r7}    @ r4 read i1, r5 write i1, r6 read i2, r7 write i2

              lsr r4, r0, #16  @ separa le bitmap lettura nella parte bassa di r4
              lsl r5, r0, #16  @ cancella i 16 bit alti
              lsr r5, r5, #16  @ bitmap di scrittura nella parte bassa di r5
              lsr r6, r1, #16  @
              lsl r7, r1, #16  @ stessa procedura per bitmap lettura in r6 e scrittura in r7
              lsr r7, r7, #16  @ per la seconda istruzione

              ands r0, r4, r7  @ calcolo intersezione fra lettura I1 e scrittura I2
              bne no          @ in caso di intersezione non nulla, salta
              ands r0, r5, r6  @ calcolo intersezione fra scrittura i1 e lettura i2
              bne no          @
              ands r0, r5, r7  @ calcolo intersezione fra scrittura i1 e scrittura i2
              bne no          @
```

```

yes:   mov r0, #1      @ se arrivo qui non ci sono intersezioni
       pop {r4-r7}
       mov pc, lr

no:    mov r0, #0      @ se invece arrivo qui, ci sono intersezioni
       pop {r4-r7}
       mov pc, lr

```

Prodotto vettoriale (IP)

Codice per realizzare il prodotto fra due vettori. Prende come parametri gli indirizzi dei due vettori (in r0 e r1) e la loro lunghezza (in r2) e restituisce la somma dei valori $v1[i]*v2[i]$ per tutti gli i da 0 a $n-1$.

Codice

```

.text
.global IP
.type IP,%function

IP:   @ r0 indirizzo del primo vettore, r1 del secondo, r2 lunghezza
      push {r4,r5} @ salvo registri non temporanei utilizzati
      mov r4, #0 @ somma parziale
do:   ldr r3, [r0],#4 @ v1[i]
      ldr r5, [r1],#4 @ v2[i]
      mul r3, r3, r5 @ v1[i]*v2[i]
      add r4, r4, r3 @ sum += v1[i]*v2[i]
      subs r2, r2, #1 @ n-- e setta i flag
      bne do @ altra iterazione se non sono a 0
      mov r0, r4 @ risultato
      pop {r4,r5} @ ripristino registri salvati sullo stack
      mov pc, lr @ ritorno al chiamante

```

nota bene:

- i puntatori vengono post incrementati in modo da scorrere i vettori in modo automatico
- la mul in questo caso calcola il risultato corretto solo nel caso in cui i fattori siano “piccoli” (altrimenti avremmo overflow), devono essere da 16 bit).
- la subs setta i flag in accordo al risultato della sottrazione. In questo caso sarebbe come se facessimo una `cmp R2, #1` (al solo fine del calcolo dei flag) ma dato che + una sub mettiamo anche il risultato in un registro (r2), cosa che la cmp non fa.